Leant

a Djex-based synthesis REPL for Lean 4

an overview and tutorial, with a detailed tour of `:synth`

An **experimental** tool in active development. Interfaces, output, and capabilities change frequently.

github.com/VladimirReshetnikov/Leant July 2026.

Contents

1 Introduction

Leant is an interactive read–eval–print loop for Lean 4, in the spirit of GHCi: you type expressions and they are evaluated, you type declarations and they enter the session, and a family of colon-commands gives you type queries, documentation, search, interactive proving, and automatic *term synthesis* with machine-checked answers. The synthesis command takes up half of this document.

Lean 4 itself is normally driven from an editor: the language server shows goals and diagnostics as you edit a file. Leant complements that workflow with a conversational one. It is aimed at exploration (trying a lemma, poking at a definition, asking *is this type even inhabited?*), where the unit of work is a line, not a file.

experimental

Leant is an **experimental project in active development**. It is a research vehicle, not a supported product: commands appear, change shape, and disappear between commits; output formats are not stable; and some features are bounded approximations by design. Every transcript in this document was produced by the tool at the date on the title page and may not match the tool you build tomorrow. When this document and `:help` disagree, believe `:help`.

Leant is implemented in Haskell and speaks the backend protocol directly; term synthesis (§5) embeds the Djex synthesis library in-process.

Under the hood, Leant drives the community `leanprover-community/repl` binary over a JSON protocol: every input becomes a command against a persistent Lean environment, and every claim Leant makes, including every synthesized term, is checked by that backend before it is shown. Sessions survive backend crashes (the session history replays automatically), can be saved and restored (`:pickle`/`:unpickle`), and can be recorded as transcripts.

2 Getting started

Build the binary once with `cabal build exe:leant` (GHC 9.12.4; the vendored `lib/Djex` submodule must be initialized), then run `leant.cmd`, or the built executable directly. Leant finds the Lean backend the same way LeanInteract's cache laid it out, or accepts an explicit `--repl-exe`/`LEANT_BACKEND`. Without a Lake project it runs against the plain Lean 4 standard library, which is also the least memorable thing about it: no imports, subsecond startup.

A session begins as a calculator and escalates quietly:

session

```
λ> 2 + 2
4
λ> it * 10
40
λ> def double (n : Nat) : Nat := n + n
λ> double 21
42
λ> :type double
double : Nat → Nat
```

Expressions are tried with `#eval` and fall back to `#check`; the last evaluated value is bound as `it`, GHCi-style. Declarations (`def`, `theorem`, `inductive`, …) extend the session environment, and any `#`-command (`#eval`, `#check`, `#print axioms`) passes straight through. Multi-line input opens automatically when a line is syntactically incomplete (type a `structure` header and just keep going), and an empty line submits it.

3 The commands

3.1 Session and evaluation

`:help, :h, :?`	show the command summary
`:quit, :q`	exit
`:type EXPR, :t`	show the type of `EXPR` (`#check`)
`:info NAME, :i`	show the definition of `NAME` (`#print`)
`:load FILE, :l`	reset the session and load a `.lean` file
`:reload, :r`	reload the last loaded file
`:import MOD`	add an import (rebuilds the session)
`:imports`	list active imports
`:undo`	revert the last state-changing command
`:reset`	clear all definitions (keeps imports)
`:history`	show the session's state-changing commands
`:set OPT VAL`	a Lean `set_option` that persists in the session

3.2 Discovery

`:browse [NS]`	list declarations in a namespace, or the session's own
`:browse! NS`	…including compiler-generated auxiliaries
`:doc NAME`	show a docstring
`:search TEXT`	search declaration names, case-insensitively
`:search? TYPE`	proof search: what proves `TYPE`? (via `exact?`)

3.3 Proving and synthesis

`:prove [PROP]`	prove `PROP` interactively, tactic by tactic (without an argument: resume the last `sorry`)
`:synth TYPE`	synthesize verified terms of `TYPE`; candidates are bound as `it1` (= `it`), `it2`, …(§5)
`:set synth-engine E`	`djinn` (default) | `exference` | `both`
`:set synth-steps N`	Exference's step budget (default 4096)
`:set synth-classical B`	offer classical candidates for constructively refuted goals (`on`|`off`, default on)
`:set synth-library B`	library premises for recursive inductives (`on`|`off`, default on; §5.5)

3.4 Persistence and utility

`:transcript [F|on|off]`	record a transcript of the session
`:timestamps [on|off]`	timestamp transcript entries
`:pickle FILE`	save the environment to `FILE` (`.olean`)
`:unpickle FILE`	restore an environment
`:env`	show the backend environment id
`:time`	toggle per-command timing
`:! CMD`	run a shell command

4 Interactive proving

`:prove` turns the prompt into a tactic loop against the backend's proof-state protocol. Every line is a tactic; `:undo` pops one step; `:qed NAME` assembles the accumulated script into a theorem and replays it into the session, so the result is usable afterwards. Each displayed proof state is followed by an automatically discovered tactic suggestion that Lean has already checked. The candidate probes are shaped by the goal and its hypotheses: an `intro` suggestion names the binders it would introduce, a disjunction goal is probed with `left`/`right`, a hypothesis whose type a single step can take apart is probed with `cases` or `obtain`, and a data-typed variable the goal mentions is probed with `induction`. The search prefers a candidate that closes the goal outright and annotates the suggestion accordingly (`closes the goal`, `splits into 2 goals`); when no single tactic closes it, a second phase chains quick finishers onto the best progressing candidates (`constructor <;> omega`, or an `obtain` step chained with the `exact` it enables), so even the opening suggestion is often a complete checked proof. The chains also try `simp_all`, unfolding the definitions the goal mentions and calling in `omega` on whatever arithmetic remains, which is how an induction suggestion can arrive as a finished proof: `induction l <;> simp_all [myLen]`, or `induction n <;> simp_all [double] <;> omega`. The suggestion is advisory—it does not advance the state or enter the script—and `:suggest` reprints it without rerunning the search. Below, the very first suggestion is already a complete proof; the session ignores it and proceeds step by step instead.

session

```
λ> :prove ∀ p q : Prop, (p → q) → p → q ∧ p
⊢ ∀ (p q : Prop), (p → q) → p → q ∧ p
suggestion: exact fun p q a a_1 => ⟨a a_1, a_1⟩  (closes the goal)
⊢> intro p q h hp
p q : Prop
h : p → q
hp : p
⊢ q ∧ p
suggestion: exact ⟨h hp, hp⟩  (closes the goal)
⊢> exact ⟨h hp, hp⟩
All goals accomplished
⊢> :qed mp_and
saved: theorem mp_and : ∀ p q : Prop, (p → q) → p → q ∧ p
```

A suggestion is not limited to one step. Here the goal needs structural induction on a user-defined function; once `intro` brings the variable into the context, the suggested chain arrives as a complete verified proof:

session

```
λ> def myLen : List Nat → Nat
…>   | [] => 0
…>   | _ :: t => myLen t + 1
…>
λ> :prove ∀ l : List Nat, myLen l = l.length
⊢ ∀ (l : List Nat), myLen l = l.length
suggestion: intro l
⊢> intro l
l : List Nat
⊢ myLen l = l.length
suggestion: induction l <;> simp_all [myLen]  (closes the goal)
⊢> induction l <;> simp_all [myLen]
All goals accomplished
⊢> :qed myLen_length
saved: theorem myLen_length : ∀ l : List Nat, myLen l = l.length
```

A `sorry` in an ordinary declaration prints its goal and offers the same loop via bare `:prove`. Prove mode also cooperates with `:synth`; §5.8 shows what that buys.

5 Term synthesis: `:synth`

`:synth TYPE` answers the question *"write me a term of this type"*; read through propositions-as-types, that is *"prove this."* Sometimes it answers the stronger question *"show me that no such term exists."* It is built on the vendored Djex library, linked in-process. Djex began by merging two classic Haskell program synthesizers behind one checked synthesis foundation: **Djinn**, a complete and terminating proof search for intuitionistic propositional logic (Dyckhoff's LJT calculus) that emits programs, and **Exference**, a ranked heuristic search under explicit budgets. It has since moved well beyond both originals. It fixes a long list of their bugs; the worst of them let a search that had merely approximated a type report that type uninhabited, and an exhausted search over an approximated space now counts for nothing. It implements much better type-class support, with a unified, validated constraint contract shared by both engines: Exference resolves givens, superclasses, and instances, while Djinn checks contexts and synthesizes dictionary-independent terms. And it adds a growing level of support for rank-N and impredicative types: goal-side quantifiers open through polarity-aware plan families, quantified hypotheses are instantiated at a bounded set of sequent-supplied types, and impredicative instantiation is admitted under a guard that never invents a polytype the query did not supply. That scope is still being improved, and Leant keeps the boundary to the engines narrow on purpose, so each improvement arrives by version bump. The engines speak Haskell types; Leant translates Lean goals into their fragment and their answers back into Lean.

Three rules run through the design:

- **The engine is never trusted.** Every candidate is re-elaborated by the Lean backend against the exact goal (`example : (T) := term`) before you see it. A rendering bug or an engine bug costs a dropped candidate, never a wrong answer.
- **Refusals come with reasons.** A goal outside the supported fragment is turned away with a note saying what fell outside, not quietly mangled into something answerable.
- **Negative verdicts are labeled by strength.** "Provably uninhabited" appears only when the translation was complete and lossless; anything weaker is reported as "no term found within bounds," which claims nothing.

5.1 Higher-order plumbing

The sweet spot is the structural fragment: arrows, products, sums, ⊥, ⊤, negation, `Iff`, and quantifiers over opaque variables. These are the "plumbing" terms one writes constantly. Free capital identifiers are auto-bound, so quick queries stay quick:

session

```
λ> :synth ((A → B → C) → (A → B) → A → C)
  it1  fun f g x => f x (g x)
λ> :synth (A × (B ⊕ C) → (A × B) ⊕ (A × C))
  it1  fun ⟨x, y⟩ => match y with | .inl z => .inl ⟨x, z⟩ | .inr w => .inr ⟨x, w⟩
λ> :synth (∀ p q : Prop, (p ↔ q) → ¬p → ¬q)
  it1  fun _ _ ⟨_, f⟩ k x => k (f x)
```

The first is the S combinator; the second distributes a product over a sum; the third is contraposition across an `Iff`, with the backward implication projected out by the pattern `⟨_, f⟩`. Binders are named by role (functions `f g h`, values `x y z`, negations and continuations `k`), a small thing that makes large candidates much easier to read.

Candidates are ranked smallest-first and *bound into the session* as `it1`, `it2`, …, with bare `it` the best one. They are ordinary definitions; you can evaluate them immediately:

session

```
λ> :synth (a → a → a)
  it1  fun _ x => x
  it2  fun x _ => x
λ> #eval it2 "left" "right"
"left"
```

Both inhabitants of $a \to a \to a$ that matter, and picking one is just using its name. For programs, as opposed to proof-irrelevant propositions, the choice between candidates is the whole point.

An `Iff` goal is treated as a pair of implications, so both directions of a logical identity arrive in one anonymous constructor. De Morgan and the absorption law:

session

```
λ> :synth (∀ p q : Prop, ¬(p ∨ q) ↔ ¬p ∧ ¬q)
```

```
  it1  fun _ _ => ⟨fun k => ⟨fun x => k (.inl x), fun y => k (.inr y)⟩, fun ⟨k1, k2⟩ z
  ↪  => match z with | .inl w => k1 w | .inr x1 => k2 x1⟩
λ> :synth (∀ p q : Prop, p ∧ (p ∨ q) ↔ p)
  it1  fun _ _ => ⟨fun ⟨x, _⟩ => x, fun y => ⟨y, .inl y⟩⟩
```

5.2 Programs you already know

Some types have one sensible inhabitant, and asking for it by type is quicker than remembering which library corner it lives in. Take the bind of the state monad:

session

```
λ> :synth ((S → A × S) → (A → S → B × S) → S → B × S)
  it1  fun f g x => match f x with | ⟨a, b⟩ => g a b
  it2  fun f g x => match f x with | ⟨a, _⟩ => g a x
  it3  fun f g x => match f x with | ⟨a, b⟩ => (match g a x with | ⟨c, _⟩ => ⟨c, b⟩)
```

The first candidate threads the state correctly. Now look at the second: it is type-correct and runs `g` on the *initial* state, throwing away whatever `f` did to it. That is the classic state-threading bug, and the type admits it just as happily. Types alone cannot tell these apart, which is why all candidates are shown and each is one keystroke from a test run. Continuation-passing style works too; this is the bind of the continuation monad:

session

```
λ> :synth ((A → R) → R) → (A → (B → R) → R) → (B → R) → R
  it1  fun f g h => f (fun x => g x h)
```

With the inductive expansion of §5.4 the same game extends to data. Here is `Option.bind`, with the lazy `.none` candidate ranked first and the real one second. When in doubt, run one:

session

```
λ> :synth (∀ a b : Type, Option a → (a → Option b) → Option b)
  it1  fun _ _ _ _ => .none
  it2  fun _ _ x f => match x with | .none => (.none) | .some y => f y
```

5.3 Impossibility, proved

When Djinn's complete search exhausts a fully translated goal, failure is a theorem:

session

```
λ> :synth (∀ a b : Type, a → b)
provably uninhabited — no closed term of this polymorphic type exists
```

No Lean tactic gives you this answer (a failing `exact?` proves nothing). Note the careful wording: the verdict is about *closed terms of the polymorphic type*. It does not say any

particular instance $a \to b$ is empty (choose $a = b$ and it isn't), and for propositions it is about *constructive* provability, a distinction §5.6 exploits. When the translation had to hide structure behind an opaque atom, the verdict backs off to match:

session

```
λ> :synth (∀ a : Type, Sigma (fun _ : Nat => a))
no term found within bounds (opaque atoms `Nat` hide structure the engine cannot
↪  analyze — not a refutation)
```

5.4 Inductive types

A non-recursive, non-indexed inductive or structure applied to all its parameters is expanded into a generalized sum of products: constructors become introduction rules, case analysis the elimination rule. This covers the standard library's little workhorses —

session

```
λ> :synth Ordering
  it1  .lt
  it2  .eq
  it3  .gt
λ> :synth (∀ a : Type, Option (Option a) → Option a)
  it1  fun _ _ => .none
  it2  fun _ x => match x with | .none => (.none) | .some y => y
λ> :synth (∀ e a b : Type, Except e a → (a → b) → Except e b)
  it1  fun _ _ _ x f => match x with | .error y => .error y | .ok z => .ok (f z)
```

— `Option.join` and `Except.map` synthesized rather than remembered. It extends to `Type`-valued classes-as-data like `Decidable`, where the eliminations write themselves, up to and including the instance combinators a library author would otherwise write by hand:

session

```
λ> :synth (∀ p : Prop, Decidable p → Decidable (¬ p))
  it1  fun _ x => match x with | .isFalse k => .isTrue k | .isTrue y => .isFalse (fun
  ↪  k1 => k1 y)
λ> :synth (∀ p q : Prop, Decidable p → Decidable q → Decidable (p ∧ q))
  it1  fun _ _ x y => match x with | .isFalse k => .isFalse (fun ⟨z, _⟩ => k z) |
  ↪  .isTrue w => (match y with | .isFalse k1 => .isFalse (fun ⟨_, x1⟩ => k1 x1) |
  ↪  .isTrue x2 => .isTrue ⟨w, x2⟩)
λ> :synth (∀ p q : Prop, Decidable p → Decidable q → Decidable (p → q))
  it1  fun _ _ x y => match x with | .isFalse k => .isTrue (fun z => absurd z k) |
  ↪  .isTrue w => (match y with | .isFalse k1 => .isFalse (fun f => k1 (f w)) |
  ↪  .isTrue x1 => .isTrue (fun _ => x1))
```

The last two are decidability of conjunction and of implication, assembled by case analysis on both instance arguments: four branches each, every branch carrying its own little proof.

Session-declared types participate the moment you declare them, and refutations over expanded inductives remain sound, because the engine saw the complete constructor list:

session

```
λ> structure Pair (A B : Type) where
…>   fst : A
…>   snd : B
…>
λ> :synth (∀ a b : Type, a → b → Pair a b)
  it1  fun _ _ x y => ⟨x, y⟩
λ> :synth (∀ a b : Type, Pair a b → Empty)
provably uninhabited — no closed term of this polymorphic type exists
```

Recursive types cannot be expanded; elimination is where undecidability lives. Their *constructors* are still sound introduction rules, though, so building is possible even where analysis is not:

session

```
λ> :synth Nat
  it1  Nat.zero
  it2  Nat.succ Nat.zero
λ> :synth (∀ a : Type, a → List a)
  it1  fun _ _ => List.nil
  it2  fun _ x => List.cons x List.nil
```

5.5 Recursion from the library

`:synth` will never invent a `Nat.rec`-based program, but for the everyday recursive types it does not have to: the library already wrote the recursion. A goal that mentions `List` or `Nat` brings a curated handful of library functions with it — `List.map`, `List.foldr`, `List.append`, `List.flatten`, `Nat.add` — instantiated at the goal's own types and offered to the engine as extra premises. Asking for a function *by its type* then works for types whose one sensible inhabitant is recursive:

session

```
λ> :synth ((a → b) → List a → List b)
  it1  List.map
  it2  fun _ _ => List.nil
λ> :synth ((a → b → b) → b → List a → b)
  it1  List.foldr
  it2  fun _ x _ => x
  it3  fun f x y => List.foldr f x y
λ> :synth (List (List a) → List a)
  it1  List.flatten
  it2  fun x => List.flatten x
```

(Trailing candidates and the truncation notes are elided here.) The library search runs beside the constructor search of the previous section, in a mode where the recursive occurrences are sealed atoms: without `nil`/`cons` in play, every candidate must route through the goal's own arguments and the offered functions, which is what keeps `List.map` ahead of the endless supply of closed terms that ignore the input. Both searches' survivors are shown together, library candidates first; `:set synth-library off` restores the constructors-only

behavior. As always, the premises are only *offers*: every candidate is re-elaborated by the backend, so a useless premise costs search time, never a wrong answer, and negative verdicts never come from the library run at all.

5.6 Classical candidates

Constructive logic goes further than newcomers expect, and `:synth` stays inside it whenever it can. Triple negation collapses, and contraposition holds in one direction, with no axioms in sight:

session

```
λ> :synth (∀ p : Prop, ¬¬¬p → ¬p)
  it1  fun _ k x => k (fun k1 => k1 x)
λ> :synth (∀ p q : Prop, (p → q) → ¬q → ¬p)
  it1  fun _ _ f k x => k (f x)
```

Peirce's law is different: it has no constructive inhabitant, and `:synth` can prove that. But rather than stopping at the refutation, it then asks the classical question, and answers with a term whose spelling shows what was used:

session

```
λ> :synth (∀ p q : Prop, ((p → q) → p) → p)
  it1  fun _ _ f => match Classical.em _ with | .inl x => x | .inr k => f (fun y =>
  ↪  absurd y k)
λ> :synth (∀ p : Prop, ¬¬p → p)
  it1  fun _ k => match Classical.em _ with | .inl x => x | .inr k1 => absurd k1 k
```

These are the proofs a human would write: one case split on excluded middle, one `absurd` where the contradiction lands. Less famous gaps between the two logics come out the same way. Weak excluded middle, and the Gödel–Dummett formula:

session

```
λ> :synth (∀ p : Prop, ¬p ∨ ¬¬p)
  it1  fun _ => match Classical.em _ with | .inl x => .inr (fun k => k x) | .inr k1 =>
  ↪  .inl k1
λ> :synth (∀ p q : Prop, (p → q) ∨ (q → p))
  it1  fun _ _ => match Classical.em _ with | .inl x => (match Classical.em _ with |
  ↪  .inl _ => .inr (fun _ => x) | .inr k => .inr (fun y => absurd y k)) | .inr k1 =>
  ↪  .inl (fun z => absurd z k1)
```

Two searches stand behind this. First, one excluded-middle assumption per atomic subformula is handed to the intuitionistic engine — complete for the propositional fragment, since intuitionistic logic plus atom instances of $p \vee \neg p$ *is* classical propositional logic. The findings are rendered as `Classical.em` case splits. If that misses, the goal's double-negation translation runs instead (complete by Glivenko's theorem) and the candidate is wrapped in `Classical.byContradiction`. Both kinds are verified like any other candidate. The whole behavior sits behind a switch:

session

```
λ> :set synth-classical off
synth classical: off
λ> :synth (∀ p : Prop, p ∨ ¬ p)
provably uninhabited — no closed term of this polymorphic type exists
(constructively — a classical proof may still exist; this is not a disproof of the
↪  proposition)
```

5.7 Dependent formulas as cargo

Leant's translation makes no attempt to analyze dependent types. It does not refuse them either: a dependent subformula becomes an opaque *atom*, compared up to α-equivalence, which the engine can carry from one side of a goal to the other. Transport works; analysis does not:

session

```
λ> opaque P : Nat → Prop
λ> opaque Q : Prop
λ> :synth ((∀ n : Nat, P n) ∧ Q → Q ∧ (∀ n : Nat, P n))
  it1  fun ⟨x, y⟩ => ⟨y, x⟩
```

The engine never looked inside $\forall n,\ P\,n$; it swapped a sealed box. A goal that would require opening the box (an induction, a rewrite, a case split on an index) is refused with a reason; that work belongs to `:prove`.

5.8 Synthesis inside a proof

Bare `:synth` in prove mode targets the current goal *with its hypotheses as premises*. Candidates are offered as `it1`, `it2`, … exactly as in the session, except that here `itN` splices the candidate applied to your hypotheses, so `exact it1` closes the goal. The hypotheses can carry real weight; below, the case analysis on the `Decidable` instance comes from `hd`:

session

```
λ> :prove ∀ p q : Prop, Decidable p → (p → q) → (¬p → q) → q
⊢ ∀ (p q : Prop), Decidable p → (p → q) → (¬p → q) → q
⊢> intro p q hd hpq hnq
p q : Prop
hd : Decidable p
hpq : p → q
hnq : ¬p → q
⊢ q
⊢> :synth
(synthesizing with hypotheses p q hd hpq hnq as premises)
  it1  fun _ _ x f g => match x with | .isFalse k => g k | .isTrue y => f y
⊢> exact it1
All goals accomplished
⊢> :qed byCases
saved: theorem byCases : ∀ p q : Prop, Decidable p → (p → q) → (¬p → q) → q
```

It also slots into an ordinary multi-goal proof. Split a conjunction with `constructor` and let `:synth` deal with each half in turn. Notice the prompt tracking the number of open goals, and the projections from `h` showing up without being asked for:

session

```
λ> :prove ∀ p q r : Prop, (p → q) ∧ (q → r) → p → r ∧ (p → q)
⊢ ∀ (p q r : Prop), (p → q) ∧ (q → r) → p → r ∧ (p → q)
⊢> intro p q r h hp
p q r : Prop
h : (p → q) ∧ (q → r)
hp : p
⊢ r ∧ (p → q)
⊢> constructor
— goal 1 of 2 —
case left
p q r : Prop
h : (p → q) ∧ (q → r)
hp : p
⊢ r
— goal 2 of 2 —
case right
p q r : Prop
h : (p → q) ∧ (q → r)
hp : p
⊢ p → q
⊢2> :synth
(synthesizing with hypotheses p q r h hp as premises)
  it1  fun _ _ _ ⟨f, g⟩ x => g (f x)
⊢2> exact it1
case right
p q r : Prop
h : (p → q) ∧ (q → r)
hp : p
⊢ p → q
⊢> :synth
(synthesizing with hypotheses p q r h hp as premises)
  it1  fun _ _ _ ⟨f, _⟩ x _ => f x
⊢> exact it1
All goals accomplished
⊢> :qed chain
saved: theorem chain : ∀ p q r : Prop, (p → q) ∧ (q → r) → p → r ∧ (p → q)
```

Unlike `exact?`, which finds an *existing* lemma, this composes a new term from the goal's own material: a constructive complement to the finisher tactics, needing no premise database and no imports. The division of labor that emerges is pleasant: the genuinely dependent moves (quantifiers, induction, rewriting) stay yours, and the propositional bookkeeping between them does not.

5.9 Engines and budgets

The default engine is Djinn: complete, terminating, and the only source of refutations. `:set synth-engine exference` switches to the ranked heuristic search, and `both` runs the two together — Djinn's candidates first, Exference's new ones appended, negative verdicts only

from the engine entitled to them. Exference runs under explicit budgets (`:set synth-steps`, default 4096), and truncation is always reported:

session

```
λ> :set synth-engine both
synth engine: both
λ> :synth ((A → B → C) → (A → B) → A → C)
  it1  fun f g x => f x (g x)
note: exference: search truncated: step limit reached, queue limit pruned 31990
```

The pure searches answer in microseconds to milliseconds; the cost center is backend verification, a few hundred milliseconds per candidate. A wall-clock guard (default 20 s, `LEANT_SYNTH_TIMEOUT`) covers the quantified goals whose bounded instantiation can widen the space; hitting it is reported as "no answer," never as a verdict.

5.10 What `:synth` will not do

Some plain statements of present limits. It will not synthesize recursion or induction; recursive types contribute constructors only. It will not analyze dependent structure; atoms are transported, never opened. Quantifier handling beyond the propositional fragment follows Djex's bounded rank-N and guarded impredicativity rules, which are sound and terminating over an undecidable space and widen as Djex improves; until then, some higher-rank goals answer "no term found within bounds" and nothing stronger. And although Djex can resolve type-class evidence itself, Leant leaves instance arguments to Lean's elaborator during verification, which sits nearer the instance tables anyway. The fragment is bounded on purpose: inside it, answers are fast, verified, and, for refutations, final.

6 How it works, briefly

A `:synth` query passes through five checked stages. A Lean metaprogram (compiled once into a cached side environment) elaborates the goal and serializes it into the engines' fragment: structural connectives are decomposed; qualifying inductives expand into constructor lists; everything else becomes an α-normalized opaque atom. The fragment translator either accepts the result or refuses with a reason. The selected engine searches; its candidates are rendered back into Lean syntax, with constructor names restored, binders named by role, and anonymous-constructor and leading-dot spellings preferred over fully qualified fallbacks. Finally the backend re-elaborates each candidate against the original goal, and only survivors are shown and bound. The engine boundary is narrow by design: the searches behind it are swappable, and nothing they emit is believed until Lean has type-checked it.

7 Status

Leant is a working prototype under active development. The synthesis design and its phased history are documented in `docs/SYNTHESIS_PROPOSAL.md`; transcript-level golden tests live in `test/`. The project began inside the *ProveIt* repository and was split out with its history in-

tact. Expect sharp edges: the project exists to find out how far a conversational, verification-gated synthesis workflow can be pushed, and it moves at the speed of that question.